

Dynamic Programming: Theory and Computation

John Stachurski

Topics

- DP problems
- DP theory
- DP algorithms

The focus of this lecture will be on

1. core theory
2. core algorithms and
3. numerical methods

For now we will stick to discrete problems

- finite action space A
- finite state space X

On the other hand, we will allow many interesting features

- Epstein-Zin preferences
- state-dependent discounting
- risk-sensitive preferences
- quantile preferences
- nonlinear discounting, etc.

Prelude: some linear algebra

In what follows we sometimes need to deal with equations of the form

$$v(x) = \sum_{x'} v(x')K(x, x') + b(x) \quad (x \in X)$$

Here

- K and b are given, while v is the unknown
- v maps finite set X to $\mathbb{R}$ (we write $v \in \mathbb{R}^X$)
- Our objective is to solve the equation for v

To make this simple we can think of this in terms of matrices and vectors

Replacing $x = x_i$ with i , the equation

$$v(x) = \sum_{x'} v(x') K(x, x') + b(x) \quad \text{for } x \in \mathbf{X}$$

becomes

$$v_i = \sum_j v_j K_{ij} + b_i \quad \text{for } i \in \{1, \dots, n\}$$

or

$$v = Kv + b$$

The spectral radius of K is

$$\rho(K) := \max\{|\lambda| : \lambda \text{ is an eigenvalue of } K\}$$

```
import jax
import jax.numpy as jnp

@jax.jit
def spectral_radius(K):
    eigenvalues = jnp.linalg.eigvals(K)
    return jnp.max(jnp.abs(eigenvalues))
```

The next result is the matrix analogue of

$$|a| < 1 \quad \implies \quad \sum_{i \geq 0} a^i = \frac{1}{1-a}$$

Theorem. If $\rho(K) < 1$, then

1. $I - K$ is invertible with

$$(I - K)^{-1} = \sum_{i \geq 0} K^i$$

2. and the equation $v = Kv + b$ has the unique solution

$$v^* = (I - K)^{-1}b$$

$$= \sum_{i \geq 0} K^i b$$

Prelude: Markov chains

Let

- X be a finite set, as before
- (X_t) be a stochastic process taking values in X
- P be a stochastic matrix ($P \geq 0$ and $P\mathbb{1} = \mathbb{1}$)

We say that (X_t) is **P -Markov** if

$$\mathbb{P}\{X_{t+1} = x' \mid X_t = x\} = P(x, x')$$

for all $x, x' \in X$


```

def generate_markov_chain(key, P, x_0, num_steps):
    " Generate a P-Markov chain with initial condition x_0. "
    n = P.shape[0]
    chain = jnp.zeros(num_steps + 1, dtype=jnp.int32)
    chain = chain.at[0].set(x_0)

    def update(t, carry):
        key, chain = carry
        key, subkey = random.split(key)
        x = chain[t]
        y = random.choice(subkey, n, p=P[x, :])
        chain = chain.at[t + 1].set(y)
        return (key, chain)

    final_key, final_chain = jax.lax.fori_loop(
        0, num_steps, update, (key, chain)
    )
    return final_chain

```

Introduction to DPs: the Affine Case

We consider dynamic programs operating on

1. a finite set X called the **state space** and
2. a finite set A called the **action space**

Actions are restricted by a **feasible correspondence** Γ

- from X to A
- $\Gamma(x)$ = actions available in state x (nonempty)

The **feasible state-action pairs** is the set

$$G := \{(x, a) \in X \times A : a \in \Gamma(x)\}$$

The **Bellman equation** takes the form

$$v(x) = \max_{a \in \Gamma(x)} \left\{ r(x, a) + \sum_{x'} v(x') K(x, a, x') \right\}$$

Components:

- the **reward function** r maps G to $\mathbb{R}$ and
- the **transition kernel** K maps $G \times X$ to $\mathbb{R}_+$

What's the intuition behind the Bellman equation

$$v(x) = \max_{a \in \Gamma(x)} \left\{ r(x, a) + \sum_{x'} v(x') K(x, a, x') \right\} \quad ?$$

Think of $v(x')$ as maximum lifetime reward from future state x'

The max trades off

1. current reward $r(x, a)$
2. expected discounted future reward $\sum_{x'} v(x') K(x, a, x')$

The result yields maximum lifetime reward $v(x)$

Example. Job search

Set up:

- wage offer process $(W_t)_{t \geq 0}$ is P -Markov on finite set W .
- discount factor $\beta \in (0, 1)$

The Bellman equation is

$$v(w) = \max \left\{ \frac{w}{1 - \beta}, c + \beta \sum_{w'} v(w') P(w, w') \right\}$$

First choice is

$$\frac{w}{1 - \beta} = w + \beta w + \beta^2 w + \dots$$

We can rewrite the Bellman equation as

$$v(w) = \max_{a \in \{0,1\}} \left\{ a \frac{w}{1-\beta} + (1-a) \left[c + \beta \sum_{w'} v(w') P(w, w') \right] \right\}$$

This is an affine DP with

- state $w \in W$
- $\Gamma(w) = \{0, 1\}$
- $r(w, a) = aw/(1-\beta) + (1-a)c$
- $K(w, a, w') = (1-a)\beta P(w, w')$

Example. Firm exit

Firm observes current state of the world z and decides whether to exit

- If exit then receive scrap value $s(z)$
- If continue then receive profit $\pi(z)$

Set up:

- state of the world $(Z_t)_{t \geq 0}$ is P -Markov on finite set Z
- discount factor $\beta = 1/(1 + r)$ with $r > 0$

The Bellman equation is

$$v(z) = \max \left\{ s(z), \pi(z) + \beta \sum_{z'} v(z') P(z, z') \right\}$$

We can rewrite the Bellman equation as

$$v(z) = \max_{a \in \{0,1\}} \left\{ as(z) + (1-a) \left[\pi(z) + \beta \sum_{z'} v(z') P(z, z') \right] \right\}$$

This is also an affine DP

- $r(z, a) = as(z) + (1-a)\pi(z)$
- $K(z, a, z') = (1-a)\beta P(z, z')$

One limitation: constant discounting

For this model we would expect

$$\beta = \frac{1}{1 + \text{interest rate}}$$

And interest rates vary over time

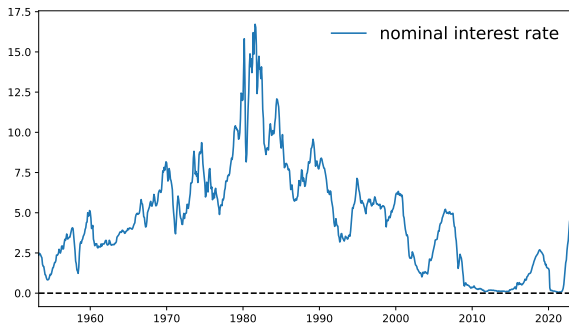


Figure: Nominal US interest rates

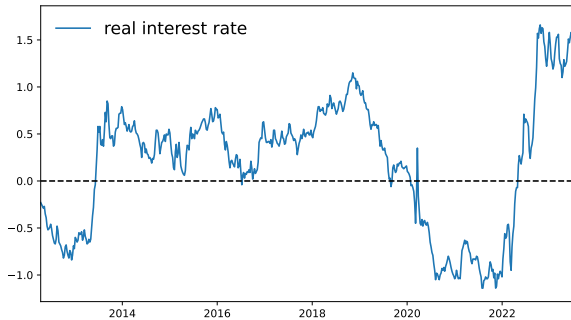


Figure: Real US interest rates

Example. Firm exit with state-dependent discounting

The Bellman equation is

$$v(z) = \max_{a \in \{0,1\}} \left\{ as(z) + (1-a) \left[\pi(z) + \beta(z) \sum_{z'} v(z') P(z, z') \right] \right\}$$

This is also an affine DP

- $r(z, a) = as(z) + (1-a)\pi(z)$
- $K(z, a, z') = (1-a)\beta(z)P(z, z')$

Example. A **shortest path problem** on graph $\mathcal{G} = (\mathbf{X}, E)$

- $c(x, x') =$ cost of traversing edge $(x, x') \in E$
- the direct successors of x denoted by

$$\mathcal{O}(x) := \{x' \in \mathbf{X} : (x, x') \in E\}$$

Aim: find the minimum cost path from x to a specified vertex d

The Bellman equation is

$$v(x) = \min_{x' \in \mathcal{O}(x)} \{c(x, x') + v(x')\}$$

This is an affine DP with

- state $x \in \mathbf{X}$ and action $a \in \mathbf{X}$
- $\Gamma(x) = \mathcal{O}(x)$
- $r(x, a) = -c(x, a)$
- $K(x, a, x') = \mathbb{1}\{x' = a\}$

Example. Inventory management

Bellman equation is

$$v(y) = \max_{a \in \Gamma(y)} \left\{ r(y, a) + \beta \sum_{y'} v(y') R(y, a, y') \right\}$$

- $y \in \{0, \dots, \bar{y}\}$ is current inventory
- a is current order (units of stock)
- $r(y, a)$ is current profits
- $R(y, a, y') = \text{prob. next state is } y' \text{ given } y, a$
- An affine DP with $K(y, a, y') = \beta R(y, a, y')$

Let's now return to the general case with Bellman equation

$$v(x) = \max_{a \in \Gamma(x)} \left\{ r(x, a) + \sum_{x'} v(x') K(x, a, x') \right\}$$

- state space is X (finite)
- action space is A (finite)
- reward function r maps G to $\mathbb{R}$
- transition kernel K maps $G \times X$ to $\mathbb{R}$
- feasible correspondence Γ is nonempty

Definitions

A **feasible policy** is a map $\sigma: X \rightarrow A$ with

$$\sigma(x) \in \Gamma(x) \quad \text{for all } x \in X$$

- $\Sigma :=$ all feasible policies

Feasible policy σ is called ***v-greedy*** if

$$\sigma(x) \in \operatorname{argmax}_{a \in \Gamma(x)} \left\{ r(x, a) + \sum_{x'} v(x') K(x, a, x') \right\} \quad \forall x \in X$$

The **Bellman operator** is

$$(Tv)(x) = \max_{a \in \Gamma(x)} \left\{ r(x, a) + \sum_{x'} v(x') K(x, a, x') \right\}$$

For each $\sigma \in \Sigma$, we introduce the **policy operator**

$$(T_\sigma v)(x) = r(x, \sigma(x)) + \sum_{x'} v(x') K(x, \sigma(x), x')$$

Note:

$$\sigma \text{ is } v\text{-greedy} \iff Tv = T_\sigma v$$

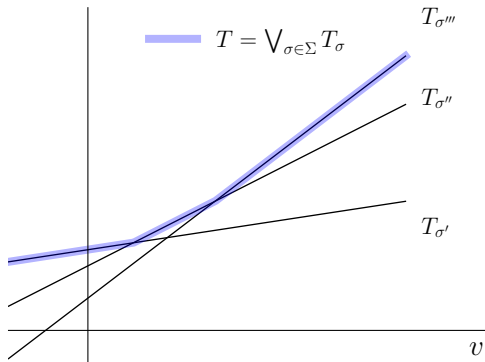


Figure: T is the pointwise max. of $\{T_{\sigma}\}_{\sigma \in \Sigma}$

Let

- $r_\sigma(x) := r(x, \sigma(x)) = \text{rewards under } \sigma$
- $K_\sigma(x, x') := K(x, \sigma(x), x') = \text{transitions under } \sigma$

Assume for now that $\rho(K_\sigma) < 1$ for all σ

The **lifetime value function** associated with policy σ is

$$v_\sigma := \sum_{t \geq 0} K_\sigma^t r_\sigma = (I - K_\sigma)^{-1} r_\sigma$$

Note that

$$v \in \text{fix}(T_\sigma) \iff v = r_\sigma + K_\sigma v$$

$$\iff (I - K_\sigma)v = r_\sigma$$

$$\iff v = (I - K_\sigma)^{-1}r_\sigma =: v_\sigma$$

Hence v_σ satisfies

$$v_\sigma(x) = r(x, \sigma(x)) + \sum_{x'} v_\sigma(x') K(x, \sigma(x), x')$$

This is a natural recursion for the lifetime value of σ

Defining optimality

We define the **value function** via

$$v^*(x) := \max_{\sigma \in \Sigma} v_{\sigma}(x) \quad (x \in X)$$

A policy σ is called **optimal** if $v_{\sigma} = v^*$

- σ extracts maximum lifetime value from all states

Optimality

Consider an affine DP with Bellman operator T and value function v^*

Theorem. If $\rho(K_\sigma) < 1$ for all $\sigma \in \Sigma$, then

1. v^* is the unique fixed point of T in $\mathbb{R}^X$
2. $T^n v \rightarrow v^*$ as $n \rightarrow \infty$ for all $v \in \mathbb{R}^X$
3. A feasible policy is optimal if and only if it is v^* -greedy
4. At least one optimal policy exists

Standard algorithms

Algorithm 1: VFI

input $v_0 \in \mathbb{R}^X$

input τ , a tolerance level for error

$\varepsilon \leftarrow +\infty$

$k \leftarrow 0$

while $\varepsilon > \tau$ **do**

$v_{k+1} \leftarrow Tv_k$

$\varepsilon \leftarrow \|v_k - v_{k+1}\|_\infty$

$k \leftarrow k + 1$

end

Compute a v_k -greedy policy σ

return σ

Algorithm 2: HPI

input $\sigma_0 \in \Sigma$, set $k \leftarrow 0$ and $\varepsilon \leftarrow 1$

while $\varepsilon > 0$ **do**

$v_k \leftarrow$ the lifetime value of σ_k

$\sigma_{k+1} \leftarrow$ a v_k -greedy policy

$\varepsilon \leftarrow \mathbb{1}\{\sigma_k \neq \sigma_{k+1}\}$

$k \leftarrow k + 1$

end

return σ_k

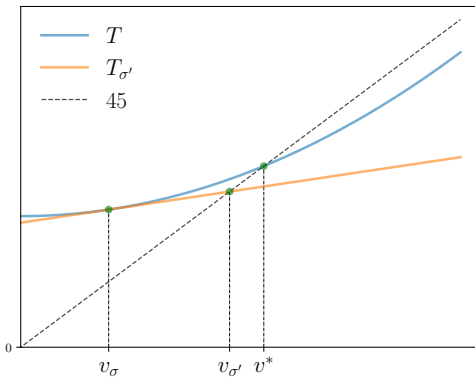


Figure: HPI as a version of Newton's method

Algorithm 3: OPI

input v_0 , an initial guess of v^*

input τ , a tolerance level for error

input $m \in \mathbb{N}$, a step size

$k \leftarrow 0$

$\varepsilon \leftarrow +\infty$

while $\varepsilon > \tau$ **do**

$\sigma_k \leftarrow$ a v_k -greedy policy

$v_{k+1} \leftarrow T_{\sigma_k}^m v_k$

$\varepsilon \leftarrow \|v_k - v_{k+1}\|_\infty$

$k \leftarrow k + 1$

end

return σ_k

Proposition. Under the stated conditions,

1. VFI, HPI and OPI all converge, and
2. HPI converges to an exact optimal policy in finitely many steps

For details and proofs see <https://dp.quantecon.org/>